

Ampel Unterhalt

Panta Rey · BTC-Ampel · Konfiguration 1.0 · Stand 18. September 2026

Was von selbst läuft, was du tun musst, und was zu tun ist, wenn etwas klemmt.

In einem Satz

Die Ampel rechnet bei GitHub, nicht in deinem Browser. Sie meldet sich bei dir, wenn etwas ist. Du musst nichts öffnen, nichts starten und nichts pflegen – ausser zwei Zahlen im Monat und einem Blick zweimal im Jahr.

1. Was von selbst läuft

Wann	Was	Ergebnis
Montag 03:10 UTC	Wochenlauf: Daten holen, Phase und Motoren rechnen, Übergänge prüfen, Backtest auffrischen	Neue <code>latest.json</code> im Repo, Seite zeigt den neuen Stand
Täglich 06:15 UTC	Tageslauf: prüft, ob eine Meldung fällig ist	Issue, falls ja
1. bis 5. des Monats	Erinnerung an die Handeingabe	Issue <i>Monatliche Handeingabe fällig</i>
1. Januar	Jahres-Review	Issue <i>Jahres-Review</i>
2. Januar	Indikator-Prüfung: driften Indikatoren?	Bericht im Repo, Issue nur wenn etwas auffällt

Jedes Issue kommt als E-Mail. Mit der GitHub-App und *Watching* auf dem Repo zusätzlich als Mitteilung aufs Handy. Die Seite ist reine Anzeige: Ob du sie täglich oder nie öffnest, ändert nichts am Ergebnis.

2. Deine Aufgaben

Wie oft	Was	Dauer
Monatlich	Zwei Zahlen ablesen und eintragen (Abschnitt 3)	5 Minuten
Halbjährlich	Lebenszeichen prüfen: läuft der Wochenlauf noch? (Abschnitt 4)	30 Sekunden
Wenn ein Signal kommt	Selbst entscheiden und selbst handeln (Abschnitt 5)	–
Einmalig, Oktober 2026	Zweiter BGeometrics-Abzug, dann Abo auslaufen lassen	20 Minuten
Nach dem Zyklusende	Konfiguration neu bewerten (Abschnitt 8)	ein Projekt

3. Monatlich: zwei Zahlen eintragen

Die Erinnerung kommt von selbst. Gebraucht werden **Angebot im Gewinn** und **STH-Realized-Price**. Sie heben die Abdeckung des Kauf-Motors von 80 auf 100 Prozent.

Schritt für Schritt

1. Auf der Ampel-Seite die **Einstellungen** öffnen.
2. Bei jeder der beiden Kennzahlen auf **Chart öffnen** tippen, den aktuellsten Wert rechts ablesen.
3. Wert eintragen, dann **Speichern**. Es erscheint ein Textblock.
4. Den Block in die Datei `data/manual.json` im Repo übertragen: Datei öffnen, Stift-Symbol, einfügen, *Commit changes*.

5. Fertig. Gerechnet wird damit beim nächsten Wochenlauf am Montag.

Wichtig: Speichern in der Oberfläche allein genügt nicht. Die Seite kann nicht ins Repo schreiben; sie bereitet den Text nur vor. Erst der Commit zählt.

Plausibilitätsprüfung ohne Chart

Angebot im Gewinn liegt als Prozentwert typischerweise zwischen 30 und 90. STH-Realized-Price ist ein Dollarbetrag in der Nähe des aktuellen Preises. Liest du versehentlich eine Nachbarlinie ab, erwischst du meist den LTH-Einstand, der deutlich tiefer liegt. Ein halbes Prozent Ableseungenauigkeit ist unerheblich; ein Faktor 100 nicht.

Nicht eintragen: Reserve Risk, RHODL-Ratio und LTH-Positionsänderung. Die Begründung steht in HANDOFF §4a. Kurz: Die Ankerpunkte von Reserve Risk sind veraltet, es würde dauerhaft Kaufzone melden; die anderen beiden heben eine Schwelle, die das Verkaufssignal kosten würde.

4. Halbjährlich: Lebenszeichen

Es gibt genau einen Fehler, den die Ampel nicht melden kann: wenn sie selbst nicht mehr läuft. Ein fehlgeschlagener Lauf schickt eine E-Mail, ein *abgeschalteter* schickt nichts. GitHub deaktiviert geplante Abläufe in Repos, in denen 60 Tage lang nichts passiert. Die Pipeline committet wöchentlich selbst und hält sich damit wach, aber Ausnahmen kommen vor.

Setz dir zwei wiederkehrende Kalendereinträge im Jahr mit dem Text: **Ampel: läuft der Wochenlauf noch?** Dann im Repo auf *Actions* schauen. Der jüngste *Wochenlauf* sollte höchstens sieben Tage alt und grün sein.

5. Wenn ein Signal kommt

Die Ampel kauft und verkauft nichts. Sie stellt fest, dass die Bedingungen erfüllt sind. Handeln musst du selbst, an deiner Börse, von Hand.

Nach dem Handeln die Tranche auf der Seite als erledigt markieren, damit die Buchführung stimmt. Entscheidest du dich dagegen, markiere sie als übersprungen. Beides wird gespeichert; ignorieren führt dazu, dass die Seite dir dieselbe Tranche weiter als offen anzeigt.

Danach fragt die Seite einmal nach einem Satz Begründung. Nimm dir die zehn Sekunden, besonders beim Überspringen. Der wichtigste Ausfallmodus des ganzen Systems ist nicht ein falscher Indikator, sondern eine nicht ausgeführte Entscheidung. Beim Zyklusende ist diese Notiz der einzige Weg, zu unterscheiden, ob das System falsch lag oder ob du ihm nicht gefolgt bist. Die Notiz landet im Journal und im Backup. „Ohne Notiz“ schliesst ab, erzwungen wird nichts.

Wenn dir ein Signal falsch vorkommt: Die Ampel ist eine Entscheidungshilfe, kein Befehl. Sie kennt vier Zyklen, mehr nicht, und alle vier sind Zyklen, in denen Bitcoin sich wieder erholt hat. Widerspricht sie dem, was du siehst, ist das ein Grund nachzudenken, nicht blind zu folgen — aber schreib den Grund auf.

Warum es keinen Stop-Loss gibt. Das ist die häufigste Kritik an so einem System und hier bewusst so. Die Kernposition soll gar nicht verkauft werden; nur der gesteuerte Teil bewegt sich, und der arbeitet über Zyklusphasen statt über Preisschwellen. Ein unangetasteter Kern plus ein begrenzter, regelgebundener Anteil, bewusst nichts dazwischen.

6. Störungen und was dagegen hilft

6.1 Ein Banner meldet eine fehlende Quelle oder Kennzahl

Woran du es merkst Gelber Balken zuoberst auf der Seite.

Warum Eine Datenquelle hat nicht geantwortet oder eine Kennzahl fehlt dauerhaft.

Was tun

Das ist **kein Defekt, sondern die gewünschte Ehrlichkeit**. Die Ampel ersetzt fehlende Werte nie durch geratene. Sie rechnet mit dem, was da ist, und sagt, was fehlt.

- Betrifft es den STH-Einstand: Wert eintragen (Abschnitt 3), sonst nichts.
- Betrifft es eine automatische Quelle und verschwindet es nach dem nächsten Wochenlauf: war eine Störung beim Anbieter, erledigt.
- Bleibt es über mehrere Wochen: Der Anbieter hat etwas geändert. Dann muss die Quelle im Code angepasst werden. Ein Abend Arbeit, keine Dauerbelastung.

Sinkt die Abdeckung unter 60 Prozent, wertet die Ampel die Woche gar nicht aus. Der Zähler für bestätigte Wochen bleibt dann stehen, statt zurückzufallen.

6.2 Ein Workflow ist rot

Woran du es merkst E-Mail von GitHub, rotes Kreuz unter *Actions*.

Warum Ein Lauf ist abgebrochen. Häufigste Gründe: Quelle nicht erreichbar, veränderte Datenstruktur, veraltete Action-Version.

Was tun

- Den Lauf öffnen, den roten Schritt aufklappen, die letzte Fehlerzeile lesen.
- Bei *timeout*, *ECONNRESET* oder ähnlichem: Anbieter war kurz weg. Oben rechts *Re-run all jobs* drücken.
- Bei einer Meldung über eine fehlende Datei oder ein unerwartetes Feld: Der Anbieter hat sein Format geändert. Das braucht eine Code-Änderung.

Ein einzelner ausgefallener Wochenlauf ist unkritisch. Die Woche wird beim nächsten Lauf nachgeholt, weil immer die volle Historie neu gerechnet wird.

6.3 Seit Wochen keine Meldung, Actions sieht leer aus

Woran du es merkst Letzter Lauf älter als sieben Tage.

Warum GitHub hat die geplanten Abläufe wegen Inaktivität abgeschaltet, oder die Workflows wurden deaktiviert.

Was tun

- Unter *Actions* den Workflow **Wochenlauf** wählen. Steht dort ein Hinweis auf Deaktivierung, auf *Enable workflow* klicken.
- Danach einmal *Run workflow* von Hand auslösen, damit der Stand aktuell wird.

Dies ist der Fall, gegen den die halbjährliche Kalendererinnerung schützt.

6.4 Die Seite zeigt einen alten Stand

Woran du es merkst Datum oben passt nicht zum letzten Wochenlauf.

Warum Der Browser zeigt eine zwischengespeicherte Fassung.

Was tun

```
Strg + F5    (Windows)
Cmd + Shift + R    (Mac)
```

- Hilft das nicht, den Knopf **Aktualisieren** oben rechts auf der Seite drücken. Er umgeht den Zwischenspeicher.
- Bleibt es dabei, unter *Actions* prüfen, ob der letzte Wochenlauf grün war. Vielleicht ist wirklich kein neuer Stand da.

6.5 Die Seite meldet „Anzeige fehlgeschlagen“

Woran du es merkst Statt der Ampel steht eine Fehlermeldung.

Warum Die Daten liessen sich laden, aber nicht darstellen. Meist eine unvollständige Datei nach einem abgebrochenen Lauf.

Was tun

- Unter *Actions* den Workflow **Wochenlauf** von Hand starten (*Run workflow*).
- Danach Strg + F5 auf der Seite.
- Bleibt der Fehler, steht die genaue Meldung im Text. Sie gehört zusammen mit der Woche in eine Notiz für die nächste Code-Sitzung.

6.6 git push wird abgewiesen

Woran du es merkst `rejected - fetch first` oder *non-fast-forward*.

Warum Die Pipeline hat selbst committet, dein lokaler Stand ist älter. Das ist Normalbetrieb, kein Fehler.

Was tun

```
git pull --rebase
git push
```

Deshalb steht in jeder Anleitung `git pull --rebase` vor dem Push. Wer es überspringt, bekommt diese Meldung.

6.7 Nach git commit erscheint ein schwarzer Editor

Woran du es merkst Bildschirm voller Tilden, unten ein Dateiname.

Warum Git hat vim geöffnet, weil die Commit-Nachricht fehlt.

Was tun

Esc drücken, dann tippen: `:q!` und Enter

Damit ist der Commit abgebrochen, die Änderungen bleiben vorgemerkt. Dann mit Nachricht wiederholen:

```
git commit -m "kurze Beschreibung"
```

Einmalig umstellen auf einen freundlicheren Editor:

```
git config --global core.editor notepad
```

6.8 node wird nicht gefunden

Woran du es merkst	Die Benennung „node“ wurde nicht als Name eines Cmdlet erkannt
Warum	Node.js ist auf diesem Rechner nicht installiert. Die Pipeline braucht es nicht, die lokalen Hilfsprogramme schon.

Was tun

- Auf nodejs.org die LTS-Version herunterladen und installieren.
- PowerShell schliessen und neu öffnen, sonst kennt es den Befehl noch nicht.

```
node --version          erwartet: v22 oder höher
```

6.9 Eine eingetragene Zahl war falsch

Woran du es merkst	Ein Indikator zeigt einen unsinnigen Wert oder springt auf Score 100.
Warum	Beim Ablesen die falsche Linie oder die falsche Einheit erwischt, etwa Menge statt Prozent.

Was tun

- In `data/manual.json` den betroffenen Eintrag korrigieren oder ganz löschen und committen.
- Beim nächsten Wochenlauf ist es bereinigt. Die Historie der übrigen Lesungen bleibt erhalten.

Vorbeugen: Der Plausibilitätstest aus Abschnitt 3 dauert fünf Sekunden.

6.10 Die Ampel schweigt einen ganzen Zyklus lang

Woran du es merkst	Über Jahre weder Kauf- noch Verkaufssignal, oder ein Zyklushoch ohne Verkaufssignal.
Warum	Möglicherweise sind die Indikatoren abgedriftet, oder das Grundmodell des Vier-Jahres-Zyklus trägt nicht mehr.

Was tun

- Unter *Actions* den Workflow **Indikator-Prüfung** von Hand starten.
- Im Bericht `reports/indikator-check.md` **Teil A** lesen, die Übersicht: Erreicht der Motor seine Zone überhaupt noch, und liegt sein höchster Wert im laufenden Zyklus über der Schwelle von Weg E2? Das ist die wichtigste Zahl des Berichts.
- **Teil B** geht die einzelnen Indikatoren durch. Urteil *verstummt* oder *ruft dauernd* heisst: dieser Indikator hat seine Aussagekraft verloren.
- **Teil C** prüft das Modell: blieb der tiefste Rückgang unter der Torschwelle von –40 Prozent? Lagen die Wendepunkte im Fenster von 9 bis 16 Monaten nach dem Hoch?

Das ist kein Notfall und nichts, was sich kurzfristig beheben liesse. Es ist Material für die Neubewertung nach dem Zyklusende. Und es gilt: Wenn die Ampel schweigt, während der Markt offensichtlich etwas anderes tut, ist das ein Grund, ihr zu misstrauen.

Bekannter Stand vom 18.09.2026: Der Verkaufs-Motor erreichte im laufenden Zyklus höchstens 37 Punkte. Weg E2 verlangt 40 bei voller Datenlage und 32 bei der heutigen Abdeckung von 80 Prozent. Das Signal von 2025 kam also zustande, mit einem Puffer von fünf Punkten. Der Kauf-Motor ist davon nicht betroffen, seine Höchstwerte liegen unverändert zwischen 79 und 96. Einzelheiten in HANDOFF §5.

6.11 Ein Indikator ruft dauernd

Woran du es merkst Issue der Indikator-Prüfung, oder eine Kachel steht über Monate auf Score 100.

Warum Absolute Ankerpunkte, die zu einer früheren Marktphase passten. Bekanntes Beispiel: Reserve Risk, seit 2022 in 100 Prozent der Wochen in Zone.

Was tun

Das ist der gefährlichere der beiden Ausfallmodi, weil er nicht schweigt, sondern den Motor konstant anhebt und der Konvergenzbedingung eine Familie schenkt.

- Kurzfristig: die betroffene Kennzahl **nicht** von Hand eintragen.
- Langfristig: Ankerpunkte nach dem Zyklusende neu schneiden. Zielgröße ist ein Anteil in Zone im einstelligen bis niedrigen zweistelligen Prozentbereich.
- Die Daten dafür liegen bereits lokal in `data/private/`. Ein zweites Abo ist nicht nötig.

6.12 Die lokalen Daten sind weg

Woran du es merkst Ordner `data/private/` fehlt nach einem Rechnerwechsel oder einem frischen Clone.

Warum Der Ordner ist bewusst von `.gitignore` ausgeschlossen und kommt mit dem Repo nicht mit.

Was tun

Betrifft nicht den Betrieb. Die Ampel läuft ohne diese Dateien; sie sind Archiv und Auswertungsmaterial.

- Kerzenarchiv jederzeit kostenlos neu holen:

```
node scripts/local/fetch-kerzen.mjs
node scripts/local/fetch-kerzen.mjs --step=86400
```

- Der BGeometrics-Abzug ist **nicht** wiederbeschaffbar, ohne erneut ein Abo zu lösen. Deshalb gehört er auf eine externe Platte. Es sind rund zwei Megabyte.

6.13 Die Seite ist gar nicht erreichbar

Woran du es merkst 404 oder Zeitüberschreitung beim Öffnen der Adresse.

Warum GitHub Pages ist gestört, oder die Veröffentlichung wurde umgestellt.

Was tun

- Auf `githubstatus.com` nachsehen, ob GitHub selbst ein Problem meldet.
- In den Repo-Einstellungen unter *Pages* prüfen, ob die Quelle noch stimmt.

Die Daten sind davon nicht betroffen. Sie liegen als Dateien im Repo und lassen sich dort auch ohne die Seite lesen.

7. Was du niemals tun solltest

Nicht tun

Warum

An `config/engine.json` drehen Die Konfiguration 1.0 hat vier Backtests, eine Empfindlichkeitsprüfung und den BGeometrics-Nachlauf hinter sich. Sie bleibt bis zum Zyklusende eingefroren. Wer mitten im Zyklus nachjustiert, optimiert auf das, was gerade passiert.

Schwellen anpassen, weil ein Signal knapp verfehlt wurde Genau das ist Überanpassung. Das knappe Signal ist die ehrliche Antwort.

Fehlende Werte schätzen und eintragen Die Ampel unterscheidet zwischen „kein Wert“ und „Wert vorhanden“. Ein geratener Wert wird wie ein gemessener behandelt.

Den BGeometrics-Token in einem Workflow verwenden

Die Nutzungsbedingungen decken das nicht, unabhängig vom Tarif. Die lokalen Programme laufen ausschliesslich auf deinem Rechner.

Rohdaten aus data/private/ ins Repo legen

Das wäre Weiterverbreitung fremder Daten. Veröffentlicht werden nur Berichte, also Kennzahlen über die Daten.

8. Der eine Termin, der zählt

Nach dem Ende des laufenden Zyklus, also etwa beim nächsten Zyklustief, steht die Neubewertung an. Das ist der einzige Punkt, den niemand automatisieren kann. Drei Dinge liegen bereits vorbereitet:

- **Weg E2 neu denken.** Drei unabhängige Prüfungen zeigen auf dieselbe Stelle: die Kopplung der Schwelle an die Datenabdeckung.
- **Ankerpunkte von Reserve Risk neu schneiden.** Sie stammen aus einer Epoche, die um 2015 endete.
- **Entscheiden, ob RHODL und LTH-Positionsänderung zurückkommen.** Beide haben an den letzten zwei Hochs kaum noch ausgeschlagen.

Alles Weitere steht in HANDOFF.md, Abschnitt 12.

9. Spickzettel

Zweck	Befehl
Tests laufen lassen	<code>node --test "test/**/*test.mjs"</code>
Änderungen abschicken	<code>git add -A</code> <code>git commit -m "Text"</code> <code>git pull --rebase</code> <code>git push</code>
Kerzenarchiv auffrischen	<code>node scripts/local/fetch-kerzen.mjs</code>
Indikatoren prüfen	<code>node scripts/indikator-check.mjs</code>
Wochenlauf von Hand	Repo > Actions > Wochenlauf > Run workflow

Wichtige Dateien. HANDOFF.md ist der Projektstand mit allen Entscheidungen und Begründungen. SPEC.md ist die verbindliche Spezifikation. config/engine.json sind die eingefrorenen Schwellen. data/manual.json sind deine Handeinträge. In reports/ liegen die Auswertungen.